



Computer Vision for Autonomous Vehicles

Lecture 2: 3D Geometry

Instructor: Golnaz Habibi, PhD

University of Oklahoma

Fall 2025

August 28th , 2025



Announcement

- Background survey is due today. It is mandatory!
- You will get access to the class organization (for submission) in Github by tomorrow.
- **Lab 1:** due next Wednesday Sept 3rd.
- Next week: Please install ROS2Jazzy and bring your laptops to the class next lecture

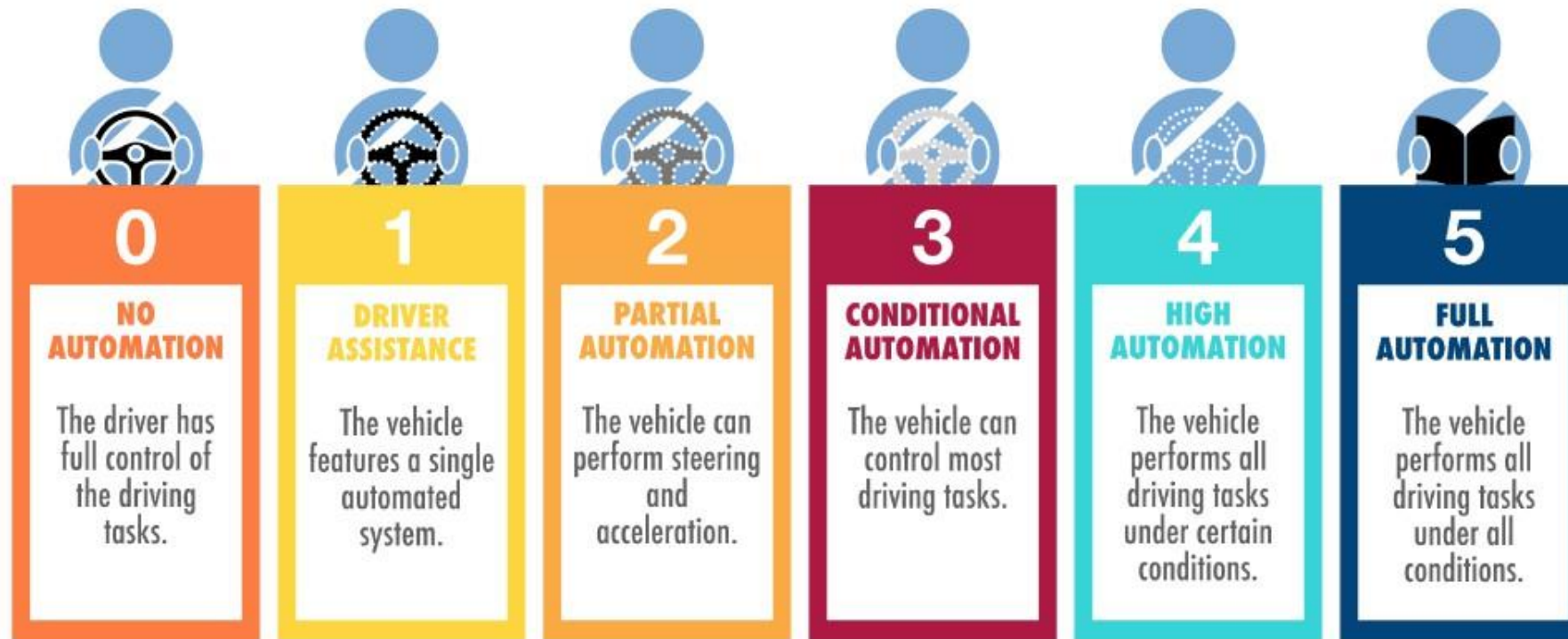
Announcement

- From lecture 1 :
 - Localization/Mapping/SLAM examples
 - Level of Autonomy
- Today lecture
 - 3D Geometry
 - Coordinate frames
 - Docker Demo (in WSL)

As a note...

- Can we reach autonomy level 5? If so, when?...
- Why autopilot in aircraft is easier than in cars??

LEVELS OF AUTONOMOUS DRIVING



Autonomous Navigation: Assumptions

- Autonomous navigation in **known** environments → the map is known)
- Autonomous navigation in **unknown** environment → a map is created as robot/car moves!)

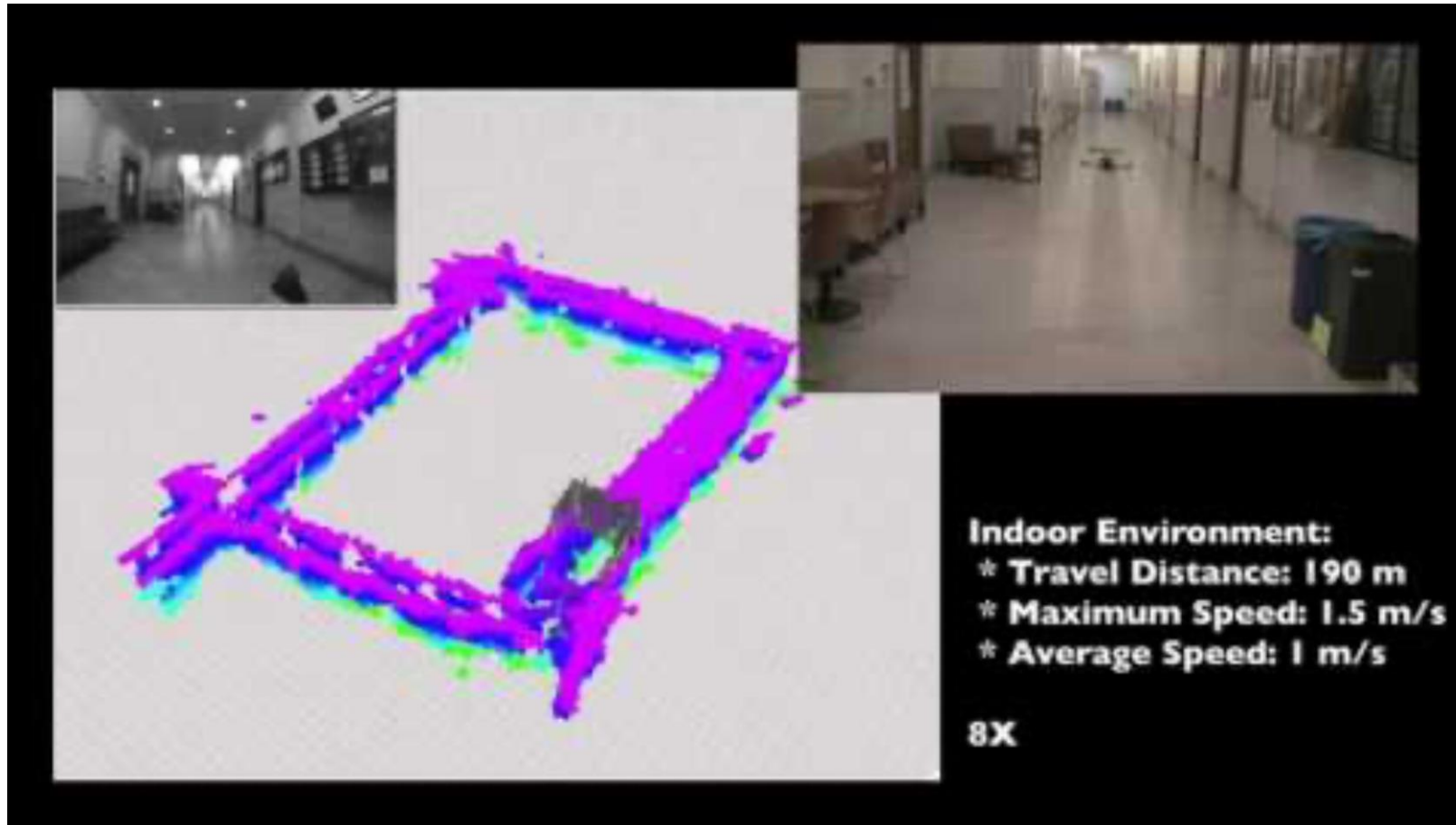
Autonomous Navigation Example in a Known Environment (pre-mapped)



Localization Vs Mapping

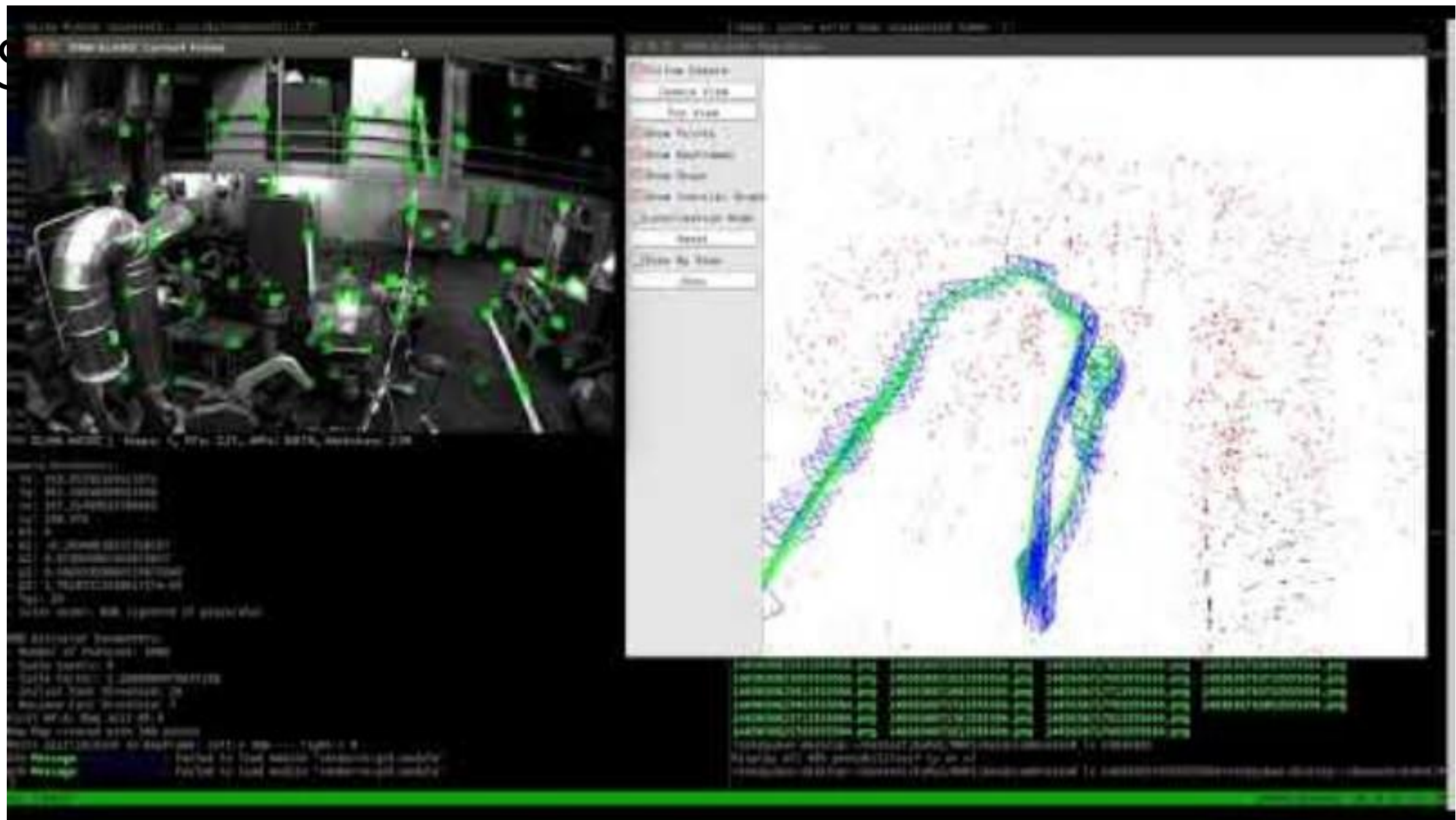
- Robot localization is the process of determining where a mobile robot is located with respect to its environment (map), to navigate in an environment
- Robotic mapping is to build the model of a robot's environment. Maps are commonly used for robot navigation.
- SLAM is Simultaneously Localization And Mapping

Mapping



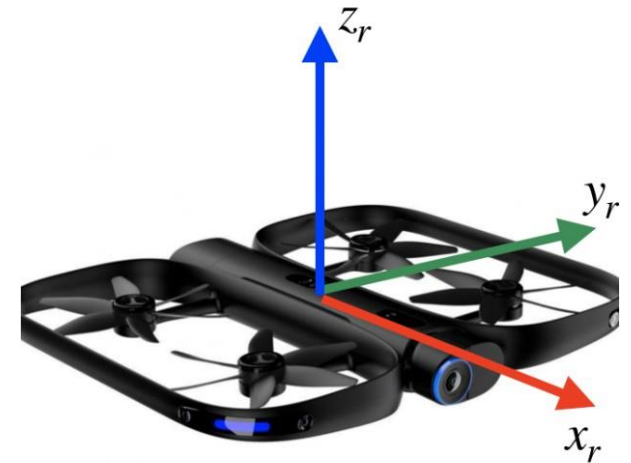
Localization (VINS Fusion)





Coordinate frame

- A coordinate frame is a set of orthogonal axes attached to a body:
- Describes position of points relative to that body.
- Origin: A single point where the axes meet.



Coordinate Frames in a Robotic System with Vision

- (x_r, y_r, z_r) Robot coordinate frame (3D):

Origin: the center of mass.

Axes: x_r forward, y_r to the left, and z_r up.

- (x_c, y_c, z_c) Camera coordinate frame (3D):

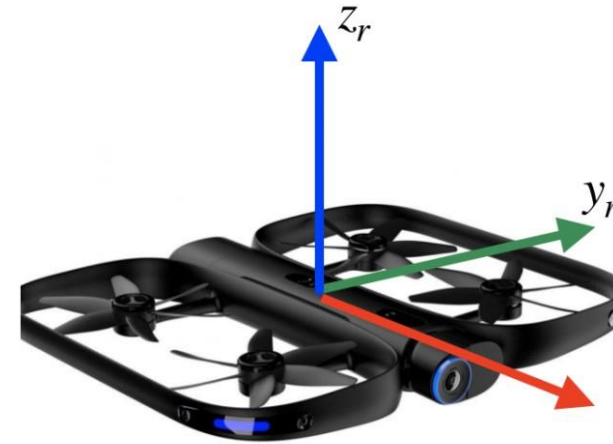
Origin: center of the camera.

Axes: x_c to the right, y_c down, z_c looking at the scene.

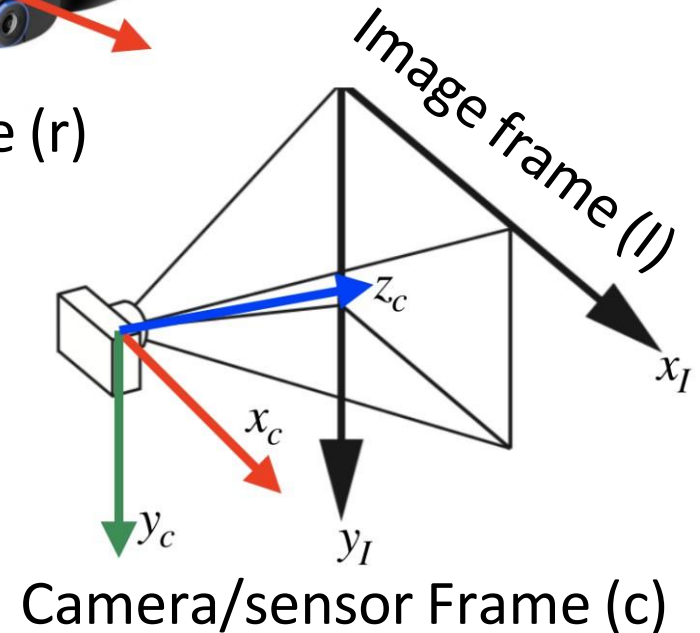
- (x_I, y_I) Image frame (2D):

Origin: top-left corner of the camera image.

Axes: looking at the camera image, x_c to the right, y_c down.



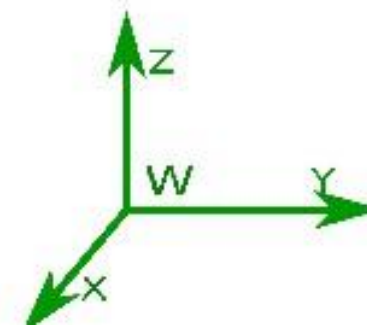
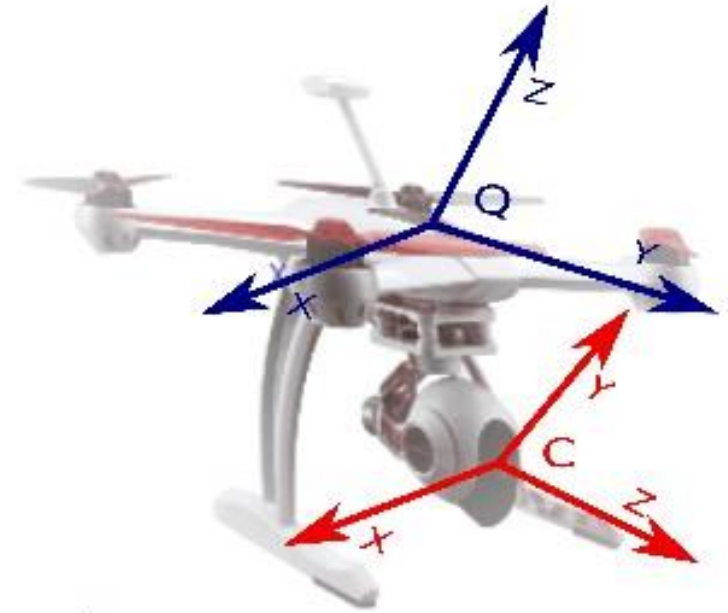
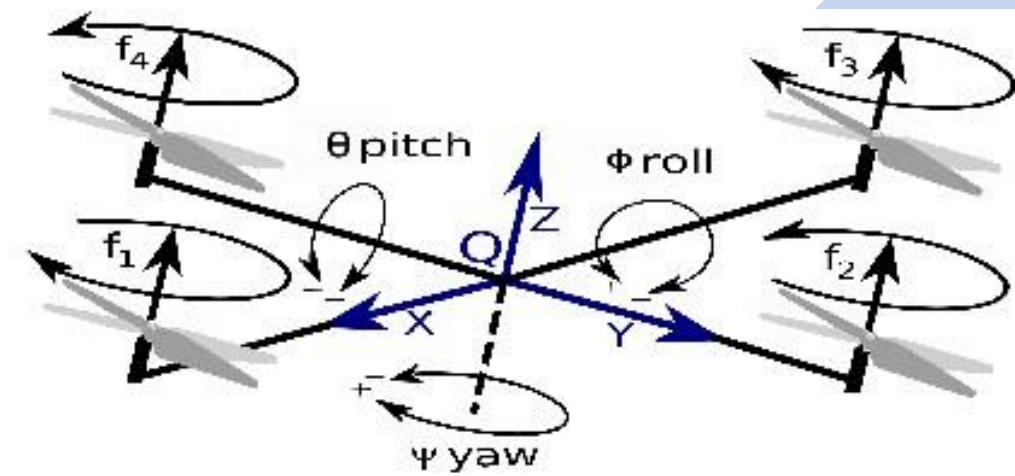
Robot Frame (r)



Camera/sensor Frame (c)

Coordinate Transform- Motivation

- Environment consists of different system in with different coordinate frame
- Robot consists of sensors, moving body, etc, each with a specific coordinate frame.
- Robots understanding from the surrounding, its decision making to navigate, is based on its coordinate frame
- We need to 'Transform' one system coordinate to another one for.
 - Navigation
 - Perception
 - Planning and control
 - Integration of different system in general



Points, positions, and translations

- Representation:

$$\mathbf{p}^w = \begin{bmatrix} p_x^w \\ p_y^w \\ p_z^w \end{bmatrix}$$

- Composition: $p_2^w = p_1^w + p_{12}^w$
- Inverse: $p_{21}^w = -p_{12}^w$

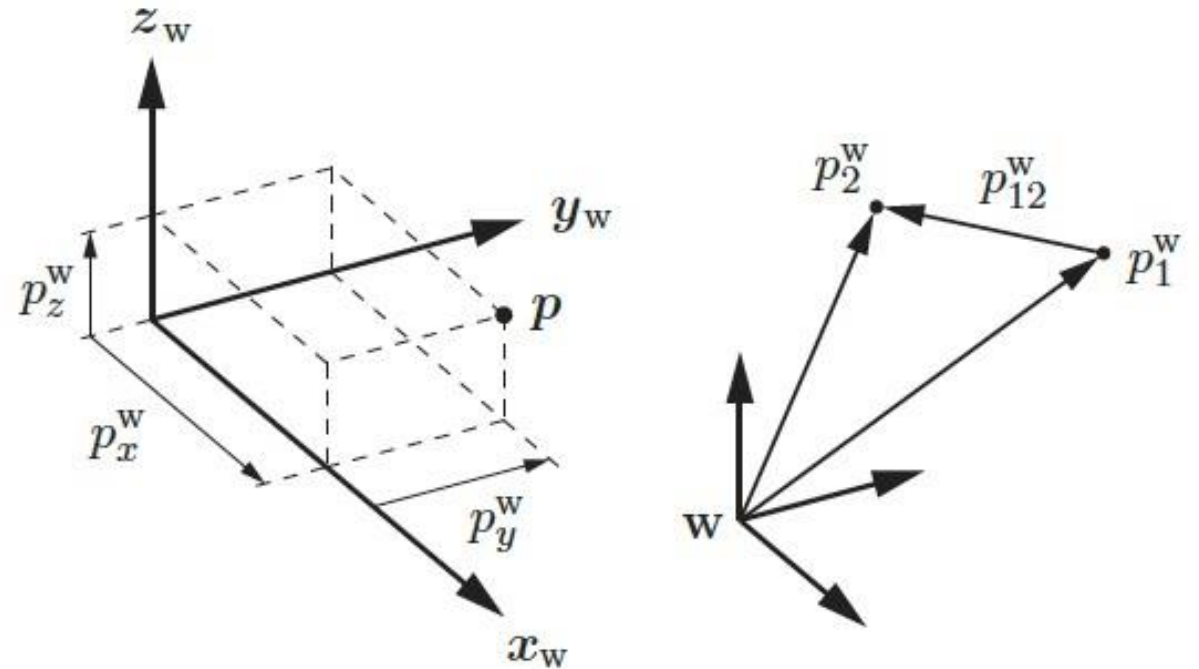
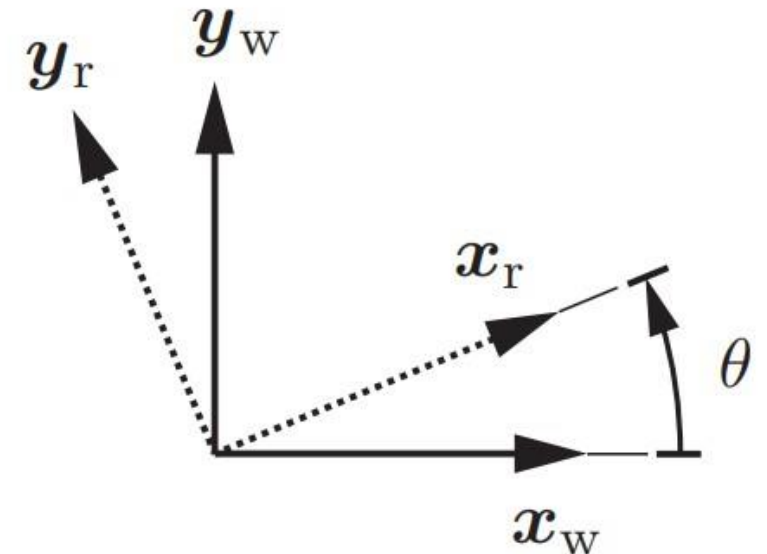


Figure 2.2: Point coordinates.

Attitude/rotation/orientation representation

- To represent the attitude of a frame r with respect to a frame w :
 - Treat the tip of each axis x_r, y_r, z_r as a point.
 - Stack the coordinates of each point with respect to the frame w as columns of a matrix.
- 2D Rotation Matrix:

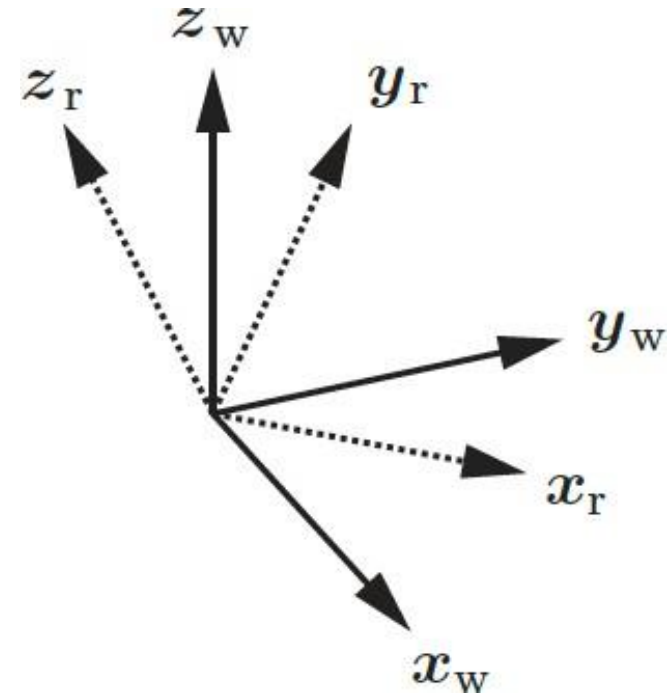
$$R_r^w = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin \theta & \cos(\theta) \end{bmatrix}$$



3D Rotation Matrix

- Each column with the of the axes of r expressed in the coordinate frame w

$$R_r^w = \begin{bmatrix} x_r^w & y_r^w & z_r^w \end{bmatrix}$$



Rotation Matrix Conditions

■ Orthogonality

orthogonality: the axes $\mathbf{x}_r^w$, $\mathbf{y}_r^w$, $\mathbf{z}_r^w$ have unit length and are orthogonal to each other (independently on the reference frame they are expressed in), therefore:

$$\begin{aligned} \|\mathbf{x}_r^w\|_2^2 &= 1 & \|\mathbf{y}_r^w\|_2^2 &= 1 & \|\mathbf{z}_r^w\|_2^2 &= 1 & \text{(unit length)} & (2.7) \\ (\mathbf{x}_r^w)^\top \mathbf{y}_r^w &= 0 & (\mathbf{x}_r^w)^\top \mathbf{z}_r^w &= 0 & (\mathbf{y}_r^w)^\top \mathbf{z}_r^w &= 0 & \text{(orthogonal vectors)} & (2.8) \end{aligned}$$

■ Right-handedness

right-handedness: the 3D axes $\mathbf{x}_r^w$, $\mathbf{y}_r^w$, $\mathbf{z}_r^w$ have to satisfy the right-hand rule, which implies that $\mathbf{x}_r^w \times \mathbf{y}_r^w = \mathbf{z}_r^w$ where $\times$ denotes the cross product between vectors. Since these are unit-length vectors, the following relations hold:

$$\mathbf{x}_r^w \times \mathbf{y}_r^w = \mathbf{z}_r^w \iff (\mathbf{z}_r^w)^\top (\mathbf{x}_r^w \times \mathbf{y}_r^w) = +1 \iff \det(\mathbf{R}_r^w) = +1 \quad (2.11)$$

Rotation Composition

- **Problem statement:** given the rotation of camera frame c respect to the robot frame (R_c^r) and the rotation of robot frame r respect to the world frame (R_r^w), what is the rotation of camera frame respect to the world frame?

$$R_c^w = ?$$

$$R_c^w = R_r^w R_c^r$$

Inverse of Rotation

- **Problem statement:** given the rotation of robot frame respect to the world frame (R_r^w), what is the rotation of world frame respect to the robot frame?

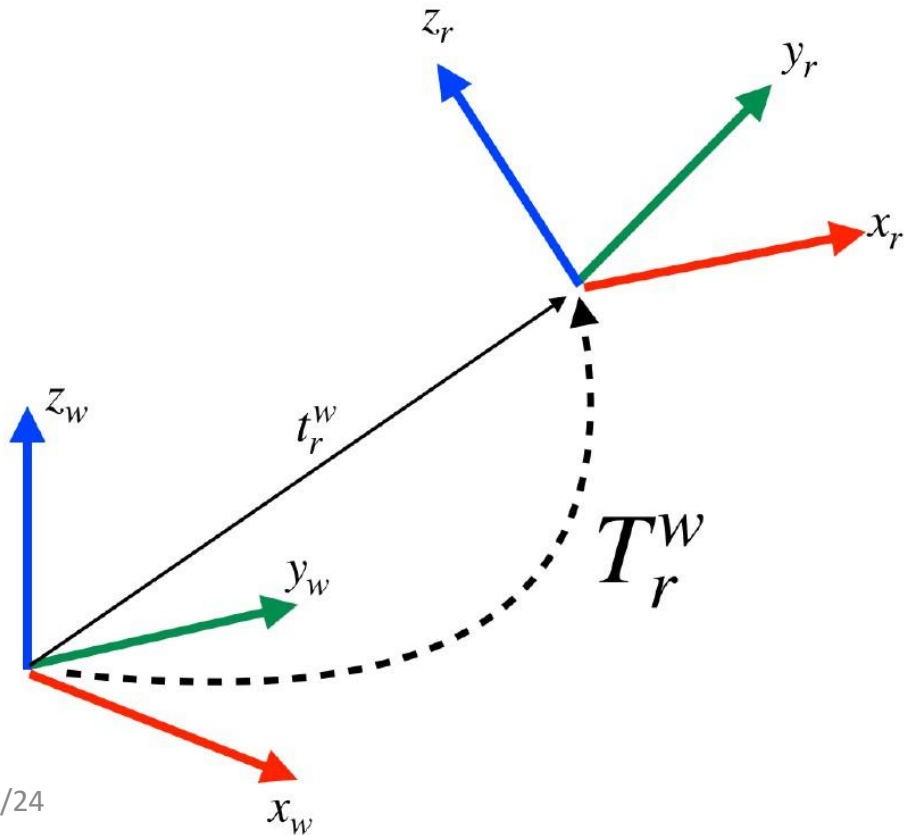
$$R_w^r = ?$$

$$\mathbf{R}_w^r = (\mathbf{R}_r^w)^{-1} = (\mathbf{R}_r^w)^T$$

Poses and rigid-body transformations

Combination of rotation and translation! $(\mathbf{R}_r^w, \mathbf{t}_r^w)$

$$\mathbf{p}^w = \mathbf{R}_r^w \mathbf{p}^r + \mathbf{t}_r^w$$



$$\mathbf{T}_r^w = \begin{bmatrix} \mathbf{R}_r^w & \mathbf{t}_r^w \\ \mathbf{0}_d^\top & 1 \end{bmatrix}$$

Homogeneous coordinate representation

$$\tilde{\mathbf{p}}^{\mathbf{r}} = \begin{bmatrix} \mathbf{p}^{\mathbf{r}} \\ 1 \end{bmatrix}$$

$$\tilde{\mathbf{p}}^{\mathbf{w}} = \begin{bmatrix} \mathbf{R}_{\mathbf{r}}^{\mathbf{w}} & \mathbf{t}_{\mathbf{r}}^{\mathbf{w}} \\ \mathbf{0}_d^{\top} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{p}^{\mathbf{r}} \\ 1 \end{bmatrix} = \mathbf{T}_{\mathbf{r}}^{\mathbf{w}} \tilde{\mathbf{p}}^{\mathbf{r}}$$

Pose composition

$$\mathbf{T}_{\mathbf{c}}^{\mathbf{w}} = \mathbf{T}_{\mathbf{r}}^{\mathbf{w}} \mathbf{T}_{\mathbf{c}}^{\mathbf{r}}$$

Inverse a pose

$$\mathbf{T}_{\mathbf{w}}^{\mathbf{r}} = (\mathbf{T}_{\mathbf{r}}^{\mathbf{w}})^{-1} = \begin{bmatrix} (\mathbf{R}_{\mathbf{r}}^{\mathbf{w}})^{\top} & -(\mathbf{R}_{\mathbf{r}}^{\mathbf{w}})^{\top} \mathbf{t}_{\mathbf{r}}^{\mathbf{w}} \\ \mathbf{0}_3^{\top} & 1 \end{bmatrix}$$